
Clase 168 — Despliegue en Vertex AI

Parte: 2 — Deep Learning · Fuente: Géron, cap. 19 § Deploying a Model to Vertex AI + docs Vertex AI. Duración estimada: 60 min.

Clase 168 — Despliegue en Vertex AI

Parte: 2 — Deep Learning · Fuente: Géron, cap. 19 § Deploying a Model to Vertex AI + docs Vertex AI.
Duración estimada: 60 min.

Objetivo

Desplegar un modelo a Vertex AI (GCP) — el servicio managed de Google para servir modelos sin mantener infraestructura. Conocer alternativas: AWS SageMaker, Azure ML, Modal, Replicate, HuggingFace Inference Endpoints.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Subir un modelo a Vertex AI Model Registry.
- Crear un Endpoint y deployar el modelo.
- Hacer requests a un endpoint Vertex AI.
- Comparar managed (Vertex/SageMaker) vs self-hosted (TF Serving + GKE/EKS).
- Conocer alternativas modernas (Modal, Replicate) para deploy serverless.

Temas

- Vertex AI Model Registry.
- Endpoint creation + traffic split (A/B testing).
- gcloud CLI: `gcloud ai models upload`, `gcloud ai endpoints deploy-model`.
- Pricing: pay per CPU/GPU hour + requests.
- Alternativas: SageMaker, Azure ML, Modal, Replicate.

Definiciones y características

- Model Registry: catálogo de modelos versionados.
- Endpoint: URL HTTP para inference.
- Traffic split: routing parcial entre versiones (A/B).
- Auto-scaling: replicas automáticas según carga.
- Prebuilt containers: TF/PyTorch/sklearn containers oficiales.

Dataset / recursos

- Modelo de clases previas exportado.
- GCP account con billing habilitado (free tier limitado).
- Librerías: `google-cloud-aiplatform`.

Ejercicios

1. Setup: `gcloud init`, `gcloud auth application-default login`. Crear bucket GCS.

2. Upload model: `gcloud ai models upload --display-name=fashion --container-image-uri=...`
3. Deploy endpoint: con `n1-standard-4`. Min/max replicas 1-3.
4. Predict request: from `google.cloud` import `aiplatform`; `ep = aiplatform.Endpoint(...)`; `ep.predict(...)`.
5. A/B traffic: deploy v2 con 20 % traffic, observar logs.

Homework verificable

Deploy del modelo Fashion-MNIST a Vertex AI:

1. Subir a GCS.
2. Upload model en Vertex Model Registry.
3. Crear endpoint y deploy.
4. Hacer 10 predicciones desde notebook local.
5. (Opcional) cleanup para no gastar.

Criterio de aceptación: predicciones llegan correctamente; cost report < \$1.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
PERMISSION_DENIED al deploy	IAM mal configurado. Fix: rol <code>aiplatform.a</code>
Endpoint queda corriendo y factura \$\$\$	Olvidaste <code>undeploy</code> . Fix: <code>gcloud ai endpoint</code>
Modelo no carga	Formato no soportado. Fix: <code>SavedModel</code> form
Predicciones lentas (cold start)	Auto-scale a 0 y arranque toma 30-60s. Fix
Output incomprensible	Signature mal definida. Fix: documentar in

Preguntas frecuentes

¿Vertex AI o SageMaker?

Depende del ecosistema. Si ya estás en AWS → SageMaker. En GCP → Vertex. Funcionalmente similares.

¿Modal / Replicate cuándo?

Para deployment rápido (serverless, pay-per-request) y para LLMs/diffusion específicamente. Modal es excelente DX, Replicate tiene modelos pre-built.

¿Self-host con K8s en lugar de managed?

Si tenés equipo de ops y volumen alto, sale más barato. Para empezar o equipos chicos, managed gana.

¿Costos típicos?

Endpoint con `n1-standard-4` 24/7 ≈ \$100/mes. Con GPU T4 ≈ \$250/mes. Si auto-scale a 0 entre requests, mucho menos.

¿Inferencia batch para datasets grandes?

`gcloud ai batch-predict-jobs create` — más barato que endpoint para volúmenes grandes sin necesidad real-time.

Referencias

- Géron, cap. 19 — Deploying a Model to Vertex AI.
- Vertex AI docs.
- SageMaker docs.
- Modal, Replicate.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Siguiente clase

Clase 169 — TF Lite (mobile/embedded)

Apéndice: notebook (primer bloque)

Primera celda ejecutable del notebook de la clase.

```
try:
    from google.cloud import aiplatform
    GCP_OK = True
except Exception:
    GCP_OK = False
    print("google-cloud-aiplatform no instalado -> se muestra la API (no se ejecuta)")

PROJECT, REGION, BUCKET = "mi-proyecto", "us-central1", "gs://mi-bucket/fashion"

if GCP_OK:
    aiplatform.init(project=PROJECT, location=REGION, staging_bucket=BUCKET)
    model = aiplatform.Model.upload(
        display_name="fashion-mnist",
        artifact_uri=BUCKET,                # SavedModel en GCS
        serving_container_image_uri=(
            "us-docker.pkg.dev/vertex-ai/prediction/tf2-cpu.2-15:latest"
        ),
    )
    print("modelo subido:", model.resource_name)
else:
    print("aiplatform.init(project=..., location=...); aiplatform.Model.upload(...)")
```

Archivos complementarios

- notebook.ipynb